Figure 1: A sample auto-created draft data entry form

have all the functions that you would usually need, ready during the Web development process. It also facilitates other functions, which can be used to query tables using conditions on any column. Let us look at a simple example—you want to create a blog, and you have two tables *'post'* and *'comment'* (in MySQL) containing columns defined as below:

```
CREATE TABLE `comment` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `post_id` int(11) NOT NULL,
  `comment_body` varchar(1000) NOT NULL,
  PRIMARY KEY (`id`),
  KEY `post_id` (`post_id`)
) ENGINE=InnoDB;

CREATE TABLE `post` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `title` varchar(128) NOT NULL,
  `body` varchar(10000) NOT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB;

ALTER TABLE `comment`
  ADD CONSTRAINT `comment_ibfk_1` FOREIGN KEY (`post_id`)
REFERENCES `post` (`id`) ON DELETE CASCADE ON UPDATE CASCADE;
```

When you run the codegen on the above schema, you get the classes *'Post'* and *'Comment'* created and available to you. You would be able to use them as shown below, to create a new blog post, for instance:

```
// Create a new blog post
$objPost = new Post();
// Initialise the database defaults to the object
$objPost->Initialize();
// Now change the values of the columns you want
$objPost->Title = "Trying QCubed";
$objPost->Body = "This is a post created using a PHP script with
the help of QCubed.";
// We do not set the 'id': it is an auto incrementing value
// Time to save
$objPost->Save();
```

Wasn't that easy? Now, what if you wanted to retrieve the post? This is how it would work, using the *Comment* class:

```
$intCommentId = 1;
$objComment = Comment::Load($intCommentId);
```

But that is not how you would want to use comments on a blog post. You would actually want to get all the comments on a particular blog post, which can be done as shown below:

```
$intBlogPostId = 20;
// Get all the comments of blog post number 20 in an array
$arrComments = Comment::LoadArrayByPostId($intBlogPostId);
// Iterate over array to print the comments one by one
foreach ($arrComments as $objComment) {
    echo "<br />" . $objComment->CommentBody;
}
```

In the above code sample, the function *LoadArrayByPostId* was created during the code-generation process, as the column *'comment.post_id'* was pointing to *'post.id'* using a foreign key, and *'comment.post_id'* did not enforce uniqueness in the column. I hope you begin to get the feel of how QCubed eases your entire query-writing process.

But that is not the end of codegen's list of advantages; it also relieves you from creating data-entry forms—a great aid to the development process! Some may instantly argue that one can use phpMyAdmin (or phpPgAdmin for PostgreSQL), but that is not the optimum solution in most cases. Allow me to explain:

- phpMyAdmin can alter the DB schema. If you happen to mistakenly alter the schema while 'entering data', you may have to begin from scratch again (depending on what you changed).
- You will not be able to use phpMyAdmin to create forms for your real website.
- phpMyAdmin would help you get things done for only MySQL. QCubed will create forms for all supported databases.
- You cannot control the execution of phpMyAdmin as long as the correct password is supplied. It will run no matter who is accessing it.

When you use codegen, you get form draft pages made for you, so you can use them to enter data into your databases straightaway! You also get ready-made Panels, which do the same job, except that you can include them (with or without modifications) in your Web pages. Well, form creation is not of great importance in QCubed, which allows you to focus on things that matter—like creating styles for the Web page, and how to add features, rather than writing SQL queries over and over again. You also get *QDataGrids* created for your tables, so searching your tables for values gets much easier. Just head to *http://examples.qcu.be/assets/_core/php/examples/*